Operating system

[CSA04A04]

CO2 - Assessment Tool - 3

Debugging

1. Deadlock

The two Process acquire the same resources in different orders

P1 : R1 → R2
P2 : R2 → R1

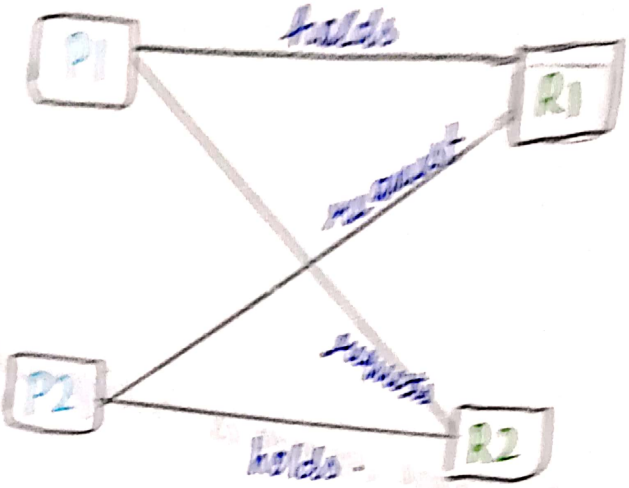
Time	Process P1	Process P2
t0	Locks R1 succeeds	
t1		Locks R2 succeeds.
t2	Waits 100 ms	waits 100 ms.
t3	tries lock(R2) → blocked	
t4		tries lock(R1) → blocked (held by P1).

Now

P1 holds R1 and waits for R2

P2 holds R2 and waits for R1

Deadlock Prevention Can Prevent, Avoiding a Circular



Optimized code -

Process P1

Lock R1

Lock R2

Critical section

Unlock (R2)

Unlock (R1)

Process P2

Lock R1

Lock R2

Critical section

Unlock (R2)

Unlock R1

Result

Both Process request resources in the same order so deadlock is eliminated.

CPU utilisation

The scheduler log shows

Ready → waiting → Ready → waiting → Ready.

Even though Processes are available, CPU utilisation remains at 30%.

Cause of the Problem

The Processes are I/O-bound. They execute for a short time on the CPU and then request I/O operations, moving to the waiting state.

Process

Ready

↓

Running
(I/O request)

↓

Waiting
(I/O completion).

↓

Ready.

Because Processes spend more time waiting for I/O than using the CPU, the Processor remains idle frequently.

Analysis

State transition	meaning
Ready - running	Process gets CPU
running - waiting	Process requests I/O
waiting - ready	I/O completes
repeated cycle.	Indicates I/O bound workload.

System-Level Optimisation.

Optimisation	Benefit
Increases multiprogramming	Another Process uses CPU while one waits for I/O
Faster I/O (SSD, caching DMA)	Reduces waiting time.
Asynchronous I/O	CPU and I/O work simultaneously.

Low CPU utilisation is caused by I/O-bound Processes repeatedly switching between ready and waiting states, increasing multiprogramming and improving I/O Performance keeps the CPU busy and increases overall system utilisation.

Memory bottleneck Debugging.

Given

Code = 5MB
Heap = 150MB
Stack = 1MB Pertthread.
Threads = 25
RAM = 2GB = 2048MB.

Calculate memory

Stack memory = 25×1
= 25MB.

Total memory used

= Code + 150+25
= 180MB.

Debugging

Although only 180 MB is used, many active threads increases Page faults and memory access overhead.

Optimization

1. Use a thread pool.
2. Improve memory locality or reduce unnecessary allocation

Result

Page faults decrease and execution becomes faster.

4. Starvation Debugging

Queue structure.

Q1 → Interactive (Highest Priority).
Q2 → System.
Q3 → Batch (P5)

Execution

Interactive jobs continuously arrive.

CPU always executes Q1

P5 remains waiting in Q3.

Root cause

Strict Priority scheduling causes starvation.

Optimization

Apply Aging

eg:- waiting time ↑
Priority ↑

P5 moves higher.

Result

P5 gets CPU time and starvation is removed.

Context switching Overhead:

Given:-

Context switched = 20,000 /sec
Time Quantum = 1ms
Burst time = 2ms

Debugging.

Thread starts
↓
Runs for 1ms
↓
Preempted
↓
Context switch. → Runs again
↓
Another context switch.

Root cause

Small time quantum causes excessive context switching

Result:

Context switches decreases and CPU efficiency improves.

6. Producer consumer debugging

Incorrect sequence :-

Producer :
wait (mutex)
wait (empty).

consumer :
wait(mutex)
wait (full).

Debugging

Suppose buffer is full.
Producer requires mutex
Producer waits on empty.
consumer needs mutex to remove an item.
Producer is waiting
Consumer is blocked.
system freeze.

Correct sequence.

Producer.
wait (empty)
wait (mutex)
insert (item)
signal (mutex)
signal (mutex).

Consumer
wait(full)
wait (mutex)
remove (item)
signal (mutex)
signal (empty).

Result
No deadlock occurs because after resource

Thread Performance debugging.

Given:-

Requests /sec = 15,000.

Thread creation = 0.4 ms

CPU = 92%.

memory continuously increases.

Debugging

Request

↓

Create thread

↓

Execute

↓

Destroy thread

This repeats thousands of times.

Root Cause

Frequent thread creation causes.

- High CPU overhead
- High memory usage.
- Increased context switching.

Optimization

- Thread pool
- Reuse worker threads.

Result

Lower CPU usage and improved scalability.

8. Banker's Algorithm.

Need Matrix

Need = maximum - Allocation

Process	maximum	Allocation	Need .
P1	(2,1,2)	(1,0,1)	(1,1,1)
P2	(1,2,1)	(0,1,0)	(1,1,1)
P3	(3,2,2)	(1,1,1)	(1,1,1)

Check request

Available = (2,1,1)

request = (1,1,1)

Temporary Available:

(2,1,1) - (1,1,1)

= (1,0,0).

(1,1,1) ≤ Need ✓

(1,1,1) ≤ Available. ✓

Safety Check

P1 needs (1,1,1) ✗

P2 needs (1,1,1) ✗

P3 needs (2,1,1) ✗ NO Process can finish.

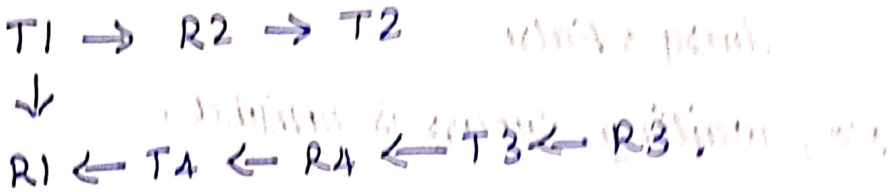
Result

The system enters a unsafe state.

Hence the request must not be granted.

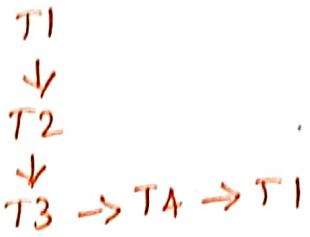
Deadlock Detection and recovery:

Resource Allocation graph:



Debugging

Cycle found:



Cycle exist
Deadlock exist.

Methods

Terminate one transaction.

Rollback one transaction.

Best methods

Rollback causes less data loss and less system distribution.

Result:

Deadlock is removed by breaking the cycle.

10. Monitor - Based Synchronization

Incorrect logic :-

busy = False

No, waiting Process is notified.

Debugging

Thread A Prints
Thread B waits
Printer becomes free
No signal is sent
Thread B waits forever.

Correct monitor :-

```
Monitor Printer {
    boolean busy = false;
    Procedure Print() {
        while (busy)
            wait();
        busy = true;
        busy = false;
        signal();
    }
}
```

Result :-

Waiting Process are awakened correctly, ensuring synchronization and fairness.